

MegaScale-Infer: Efficient Mixture-of-Experts Model Serving with Disaggregated Expert Parallelism

1 Paper summary

This paper looks at the problem of serving sparse MoE models during inference, especially in the decoding phase. The main issue is that expert modules (FFNs) are often underutilized, because each expert only processes a small part of the batch. This leads to poor GPU usage and inefficiency overall.

The authors argue that one reason for this is that attention and expert computation are too tightly coupled in existing systems. Because of this, it's harder to batch things efficiently and hardware resources are not used well.

To solve this, they propose MegaScale-Infer, which separates (or disaggregates) attention and expert computation onto different nodes. This allows them to scale independently and even run on different types of hardware, depending on cost and performance.

The system uses a pipeline approach (called ping-pong pipeline parallelism), where attention and expert nodes alternate work. There is also a deployment planner that decides things like batch size, parallelism strategy, and hardware allocation. Since this design changes the communication pattern, they also introduce a custom communication library optimized for it.

The focus is mainly on the decoding phase, where inefficiency is more visible. The results show that this approach can improve throughput and cost efficiency, even if it depends on how well the system is configured.

2 Contributions

The first contribution is the idea of separating attention and expert computation. This is important because it allows each part to scale independently and improves utilization, especially for experts that would otherwise be idle most of the time.

The second contribution is the pipeline design and deployment planning. The system carefully schedules work between attention and expert nodes, and chooses parameters like batch size and parallelism to keep everything busy. This is necessary, otherwise the disaggregated design would not work well.

The third contribution is the custom communication mechanism. Since the communication pattern is different from standard MoE systems, they design a specific solution for M2N/N2M communication. This helps reduce overhead, even if it adds more complexity to the system.

3 Problems

One limitation is the increased complexity of the system. Disaggregating components, managing pipelines, and tuning deployment parameters makes everything harder to implement and operate compared to simpler designs.

Another issue is that the benefits depend a lot on the workload and infrastructure. If communication between nodes is not fast enough, or if the workload is small, the advantages of disaggregation might be reduced. So the system works best under certain assumptions.

A third point is that the paper mainly focuses on the decoding phase. While this is an important bottleneck, it's not clear how much the approach helps for full end-to-end serving, where other factors (like prefill or system overhead) might matter more. So the results are convincing, but only for a specific part of the problem.